

Projeto 2

Algoritmos Avançados de Procura

Licenciatura em Tecnologias Digitais e Inteligência Artificial

Ano Letivo 2025/2026

Objetivos

Neste projeto pretende-se que os alunos demonstrem capacidade de:

- Modelar problemas de otimização e de procura usando diferentes paradigmas estudados nas aulas 6 a 11;
 - Implementar e comparar algoritmos de metaheurísticas, algoritmos genéticos, procura adversarial, satisfação de constrangimentos (CSP) e planeamento;
 - Analisar criticamente o desempenho dos algoritmos em termos de qualidade da solução, tempo de execução e escalabilidade;
 - Documentar e apresentar os resultados de forma clara e rigorosa.
-

Problema Obrigatório

Problema 1: Comparação CSP vs. Algoritmo Genético — O Problema das N-Rainhas

O problema das N-Rainhas consiste em colocar N rainhas num tabuleiro de xadrez $N \times N$ de modo que nenhuma duas rainhas se ataquem mutuamente. Uma rainha ataca todas as peças na mesma linha, na mesma coluna e nas mesmas diagonais.

Este problema foi já abordado no Projeto 1, onde foi resolvido usando algoritmos de procura em espaço de estados. O interesse de reutilizar o mesmo problema reside precisamente em permitir uma comparação direta entre paradigmas distintos de resolução, avaliando vantagens e limitações de cada abordagem para o mesmo domínio.

Tarefas:

1. **Recolha de Resultados do Projeto 1:** Reutilizar os resultados obtidos no Projeto 1 para o problema das N-Rainhas (procura em espaço de estados com BFS, DFS, ou outros algoritmos aí implementados). Estes resultados servirão de baseline para a comparação.
2. **Modelagem CSP:** Formular o problema das N-Rainhas como um problema de satisfação de constrangimentos. Implementar um resolvedor CSP que utilize retrocesso (*backtracking*) com verificação antecipada (*forward checking*) ou manutenção de consistência de arcos (AC-3).
3. **Modelagem com Algoritmo Genético:** Formular o problema das N-Rainhas como um problema de otimização a resolver com um algoritmo genético. Definir uma representação adequada, uma função de aptidão (*fitness*), operadores de seleção, cruzamento e mutação.

4. **Comparação Experimental:** Comparar as três abordagens (procura em espaço de estados do Projeto 1, CSP e algoritmo genético) para diferentes valores de N (pelo menos $N = 8, 16, 32, 64$). Para cada valor de N , medir e reportar:

- Tempo de execução médio (em segundos);
- Número de iterações/nós expandidos até encontrar uma solução;
- Taxa de sucesso (percentagem de execuções que encontram solução dentro de um limite de tempo definido);
- Discussão crítica das vantagens e limitações de cada paradigma para este problema.

Problemas Opcionais

Os alunos devem escolher **dois** problemas do conjunto seguinte para resolver adicionalmente.

Problema 2: O Problema do Caixeiro Viajante (TSP) — PSO e ACO

O Problema do Caixeiro Viajante (TSP) consiste em encontrar o caminho mais curto que visita um conjunto de cidades exatamente uma vez e regressa à cidade de partida.

Tarefas:

1. Implementar o algoritmo *Particle Swarm Optimization* (PSO) adaptado para resolver instâncias do TSP, definindo uma representação adequada das partículas e operadores de atualização de posição e velocidade.
2. Implementar o algoritmo *Ant Colony Optimization* (ACO) para resolver o mesmo conjunto de instâncias do TSP, definindo a representação do grafo, a regra de transição de estados e a atualização de feromonas.
3. Testar ambos os algoritmos com pelo menos três ficheiros de instâncias de dimensões diferentes.
4. Comparar o desempenho do PSO e do ACO em termos de qualidade da solução, tempo de execução e estabilidade dos resultados.
5. Analisar a influência dos parâmetros de cada algoritmo (por exemplo, fatores de inércia e aprendizagem no PSO; taxa de evaporação e peso das feromonas no ACO) na qualidade da solução e no tempo de execução.

Problema 3: O Problema do Caixeiro Viajante com Restrições (CVRP) — Algoritmos Genéticos

O CVRP (*Capacitated Vehicle Routing Problem*) é uma variante do TSP em que um veículo com capacidade limitada deve servir um conjunto de clientes a partir de um depósito.

Tarefas:

1. Implementar um algoritmo genético para resolver o CVRP.
2. Definir uma representação cromossómica adequada, operadores de cruzamento e mutação que preservem a viabilidade das soluções (ou uma penalização adequada para soluções inviáveis).
3. Testar com pelo menos duas instâncias de dimensão moderada e analisar os resultados.

Problema 4: Jogo Competitivo — Procura Adversarial

Escolher um jogo de dois jogadores com informação perfeita (por exemplo, Lig-4, Gomoku, ou um jogo à escolha do aluno com aprovação do docente) e implementar um agente inteligente.

Tarefas:

1. Implementar o algoritmo *minimax* com poda alfa-beta.
2. Definir uma função de avaliação heurística adequada ao jogo escolhido.
3. Testar o agente contra outro agente (pode ser um agente com jogada aleatória ou outro agente implementado por um colega) e analisar o desempenho.
4. Possibilitar jogar contra um utilizador humano.
5. Discutir a influência da profundidade de procura e da qualidade da função de avaliação no desempenho do agente.

Problema 5: Coloração de Mapas — Satisfação de Constrangimentos (CSP)

O problema da coloração de mapas consiste em atribuir uma cor a cada região de um mapa de modo que regiões adjacentes não partilhem a mesma cor, usando no máximo K cores.

Tarefas:

1. Formular o problema da coloração do mapa de Portugal continental (distritos) como um CSP.
2. Implementar um resolvidor CSP com retrocesso e heurísticas de seleção de variáveis (MRV — *Minimum Remaining Values*) e de valores (LCV — *Least Constraining Value*).
3. Determinar o número cromático mínimo do grafo (menor número de cores necessário) e apresentar uma coloração válida.

Problema 6: Planeamento — Mundo dos Blocos (*Blocks World*)

O Mundo dos Blocos é um domínio clássico de planeamento em que um conjunto de blocos deve ser reorganizado a partir de uma configuração inicial para uma configuração objetivo, usando operações de empilhar e desempilhar.

Tarefas:

1. Formalizar o domínio do Mundo dos Blocos em STRIPS (predicados, ações com pré-condições e efeitos).
2. Implementar um planeador (por exemplo, *Graphplan* ou planeamento por procura no espaço de estados).
3. Testar o planeador com pelo menos três problemas de complexidade crescente e analisar o desempenho.

Entregas

Cada grupo deve entregar:

1. **Código-fonte:** Todo o código implementado, organizado de forma clara e com comentários adequados. O código deve ser executável e incluir instruções de utilização.
2. **Relatório:** Um documento em formato PDF ou notebook que inclua:
 - Descrição das abordagens implementadas;

- Justificação das decisões de modelagem e dos parâmetros utilizados;
 - Apresentação e análise dos resultados experimentais (com tabelas e gráficos);
 - Discussão crítica dos resultados, incluindo limitações e possíveis melhorias;
 - Conclusões.
-

Prazo de Entrega

Data limite: 10 de junho de 2026 (inclusive).

As entregas devem ser feitas através da plataforma Moodle da unidade curricular.

Notas

- O trabalho deve ser realizado em grupos de 2 a 3 alunos.
- É permitida a utilização de bibliotecas auxiliares (por exemplo, para manipulação de grafos ou leitura de ficheiros), mas os algoritmos principais devem ser implementados pelos alunos.